

---

# Code-Konventionen für Java

---

In diesem Dokument sind die wichtigsten Code-Konventionen für die Programmierung mit Java zusammengefasst. Es beruht auf den Empfehlungen von Sun [1].

Es muss betont werden, dass es sich bei diesen Regeln um *Konventionen* handelt, die nicht Bestandteil der Syntax von Java sind.

Zielsetzungen dieser Code-Konventionen:

1. Sie schreiben Programme, die verständlich sind für die Personen, die Ihren Code lesen oder verwenden.
2. Die kohärente Namensgebung ermöglicht ein schnelleres Schreiben des Codes und macht diesen beständiger.
3. Sie werden mit den Konventionen vertraut, die in Unternehmen angewandt werden.

Im Rahmen der praktischen Arbeiten für die Module 622 und 624 sind die im vorliegenden Dokument **gelb markierten** Code-Konventionen obligatorisch.

## 1. Sprache

Verwenden Sie für Ihren Code vorzugsweise die englische Sprache. Einerseits sind die Schlüsselwörter von Java in Englisch, andererseits ist Englisch in allen Unternehmen, die sich in irgendeiner Weise mit der Herstellung von Code beschäftigen, obligatorisch.

## 2. Dateien

Java-Quelldateien haben das Suffix `.java`. Kompilierte Dateien (Bytecode) haben das Suffix `.class`.

Jede Java-Quelldatei enthält eine einzige Klasse oder öffentliche Schnittstelle. Der Name dieser Datei (ohne das Suffix `.java`) *muss* dem Namen dieser Klasse oder öffentlichen Schnittstelle entsprechen. Werden private Klassen oder Schnittstellen verwendet, muss die öffentliche Klasse oder Schnittstelle als erste in der Datei definiert werden.

Der Inhalt aller Dateien muss wie folgt organisiert werden:

- Kommentarblock (am Anfang) mit:
  - dem Kontext der Klasse, z. B. Antwort auf die Frage xyz der praktischen Arbeit der Woche 3,
  - dem Namen des oder der Autoren,
  - dem Erstellungsdatum und eventuell der letzten Aktualisierung.
- Deklaration eines eventuellen Pakets (*Package*) der Klasse, gefolgt von den eventuellen Importen:
  - `package dupont.exercises;`
  - `import java.awt.*;`
- Definition der Klasse oder der Schnittstelle (falls es mehrere gibt, wird die öffentliche Klasse oder Schnittstelle zuerst definiert). Diese Definition wird in der folgenden Reihenfolge durchgeführt:

|                                                                                                                                                                |                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Dokumentation der Klasse oder der Schnittstelle (Kommentare <code>/** ... */</code> )                                                                          | vgl. § 4                                                                                          |
| Deklaration der Klasse oder der Schnittstelle                                                                                                                  | Beispiel: <code>public class Bank</code>                                                          |
| Programmierungskommentare ( <code>/* ... */</code> ), bei Bedarf, d. h. allgemeine Kommentare zur Anwendung der Klasse, die nicht in die Dokumentation gehören | vgl. § 4                                                                                          |
| Deklaration der Klassenvariablen ( <code>static</code> )                                                                                                       | Zuerst <code>public</code> , dann <code>protected</code> , und anschliessend <code>private</code> |
| Deklaration der Instanzvariablen                                                                                                                               | Zuerst <code>public</code> , dann <code>protected</code> , und anschliessend <code>private</code> |
| Konstruktoren                                                                                                                                                  |                                                                                                   |
| Methoden                                                                                                                                                       | Vorteilhaft ist, diese nach Funktionalität zusammenzufassen                                       |

Beispiel:

```
/*
 * Exercise 622, week 3, question 2
 * Author: C. Haddock
 * Date creation: 18.02.2009
 * Date last modification: 19.02.2009
 */
```

```

package hevs.progmod;

import java.util.ArrayList;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/**
 * The <code>Mystery</code> class represents ...
 *
 * @author Captain Haddock
 * @see java.lang.StringBuilder
 */

public final class Mystery
{
    private final char value[];
    ...

```

### 3. Einrückung

Definieren Sie eine Einrückung (in der Regel 4 Leerzeichen) und verwenden Sie diese in Ihrem gesamten Code. Die Einrückungsstufe entspricht dem Verschachtelungsgrad Ihres Codes.

Seien Sie auch kohärent bei der Verwendung von geschweiften Klammern (geöffnete und geschlossene). SUN empfiehlt, die geöffnete geschweifte Klammer ans Ende der Zeile zu setzen, bei der der betreffende Block beginnt, und die geschlossene geschweifte Klammer auf eine separate Zeile mit einer Einrückung auf der Höhe der Anweisung, die den Block geöffnet hat. Methoden mit einem leeren Anweisungsblock bilden eine Ausnahme: Die geschlossene geschweifte Klammer folgt direkt auf die geöffnete geschweifte Klammer.

Beispiele:

```

class Test extends Object {
    int var1;
    int var2;

    Test(int i, int j) {
        var1 = i;
        var2 = j;
    }

    int emptyMethod() {}

    void compute() {
        ...
        if (var1 > 0) {
            var2--;
            ...
        }
    }
}

```

```

    ...
    while (var1 != var2) {
        ...
    }
}
...
}

```

Verhindern Sie zu lange Zeilen (mit mehr als 80 Zeichen):

- Ein sehr langer Ausdruck kann jederzeit über mehrere Zeilen geschrieben werden. In diesem Fall wird darauf geachtet, den Zeilenumbruch so zu wählen, dass die Verständlichkeit trotzdem gewährleistet wird (z.B. nach einem Komma, vor einem Operator, bei der umfassendsten Klammer). **Achten Sie bei Anweisungen, die über mehrere Zeilen reichen, darauf, die neue Zeile mit dem Beginn des Ausdrucks auf der vorherstehenden Zeile auszurichten.**
- Sie können die Verständlichkeit einer komplizierten Anweisung auch verbessern, indem Sie temporäre Variablen verwenden.

Verwenden Sie Ihren gesunden Menschenverstand: Wenn Sie selbst Mühe haben, eine Anweisung zu verstehen, ist dies ein Zeichen dafür, dass sie neu formatiert werden muss oder temporäre Variablen verwendet werden müssen, um die Verständlichkeit zu verbessern.

Beispiele:

```

fonc(longExpression1, longExpression2, longExpression3,
     longExpression4, longExpression5);

```

```

var = fonc1(veryLongExpression1,
           fonc2(veryLongExpression2,
                veryLongExpression3));

```

```

taxToPay = captainAge * (length + width - height)
          + 4 * shirtSize;

```

Verwenden Sie bei `if`-Anweisungen, wenn die Bedingung über mehrere Zeilen geht, eine Einrückung um 8 statt um 4 Leerzeichen, damit die Anweisung lesbar bleibt:

```

if ((condition1 && condition2)
    || (condition3 && condition4)
    || (condition5 && condition6)) {
    doSomething()    // remains ok to read
    ...
}

```

## 4. Kommentare

### 4.1. Programmierung

Man verwendet diese Kommentare entweder dazu, einen Teil des Codes, der entwickelt wird, zu „verbergen“, oder um gewisse Einzelheiten zu erklären. Sie können auf unterschiedliche Weise verwendet werden:

1. **Blockkommentare:** dienen dazu, Dateien, Methoden, Datenstrukturen und Algorithmen zu erläutern. Ihnen muss eine Leerzeile vorangestellt werden und sie weisen folgende Form auf:

```
/*  
 * This is a block of comments.  
 * It can be on several lines, each beginning with a star character.  
 * Stars are aligned in a column by using a space before and after the star.  
*/
```

2. **Zeilenkommentare:** Wenn Ihr Kommentar über eine Zeile hinausgeht, verwenden Sie nicht zwei Zeilen, sondern verwenden Sie einen Blockkommentar. Beispiel:

```
if (condition) {  
    /* Single line of comment */  
    ....  
}
```

3. **Zeilenendkommentare:** Sie können folgende Formen aufweisen: `/* ... */` oder `// ....`. Die zweite Form dient zudem dazu, einen Teil des Codes, der entwickelt wird, zu „verbergen“. Beispiele:

```
if (a == 2) {  
    return TRUE;      /* in-line block comment */  
}  
else if (toto > 1) {  
    // in-line comment using double slash  
    ...  
}  
else  
    return FALSE;     // in-line comment  
  
// if (t > 1) {  
//  
//    // this code is commented out as it need to be validated  
//    ...  
//}  
//else  
//    return FALSE;
```

## 4.2. Dokumentationskommentare

Diese Kommentare werden, wie die anderen, vom Compiler ignoriert. Sie werden jedoch vom Tool `javadoc`, mit dem HTML-Seiten zur Dokumentation Ihrer Programme erstellt werden können, interpretiert. Auf der Website von `javadoc` [2] finden Sie eine Einführung sowie ein Handbuch mit Beispielen. Ein Dokumentationskommentar beginnt mit `/**` und endet mit `*/`.

In der Regel wird empfohlen, **eine Klasse zu kommentieren** (s. Beispiel `Specialisation of the parent class Client` weiter unten) sowie alles, was öffentlich ist (Methode, Instanzvariable, Klassenvariable), da diese Elemente von „ausserhalb“ einer Klasse sichtbar sind. Diese Empfehlung ist in IT-Firmen in der Regel obligatorisch.

Beispiele für Dokumentationskommentare:

### Für eine Klasse:

```
/**
 * Specialisation of the parent class Client.
 * @author    John Smith
 * @see       Client
 */
class RetailClient extends Client
{
    ...
}
```

### Für eine Methode:

```
/**
 * Return the amount after application of the
 * value-added tax.
 * @param amountBeforeTax the amount before taxation
 * @return amount after tax
 */

public double addVAT(double amount) {
    ...
}
```

## 5. Deklarationen und Anweisungen

Es wird empfohlen, **pro Zeile nur eine Variablendeklaration** zu verwenden und die Deklarationen zu Beginn eines Blocks zusammenzufassen. Versuchen Sie, die lokalen Variablen zu initialisieren, sobald sie deklariert sind (in einigen Fällen ist dies natürlich schwierig oder unmöglich, da die Initialisierung von einer Berechnung abhängt).

Um Missverständnissen vorzubeugen, sollte das Verdecken einer Variablen in einer übergeordneten Ebene durch eine Variable gleichen Namens in einer untergeordneten Ebene vermieden werden. Beispiel:

```

int total;
...
function() {
    if (condition) {
        int total;        // avoid this!!
        ...
    }
    ...
}

```

Verwenden Sie pro Zeile nur eine Anweisung. Verhindern Sie zum Beispiel:

```
Konto++; Wert -= Konto;    // avoid this!!
```

## 6. Namen

Verwenden Sie immer aussagekräftige und eindeutige Namen. Halten Sie sich an die wichtigsten Regeln in Java: Grossbuchstaben für jedes neue Wort (`captainHaddockAge`).

| Typ            | Regeln                                                                                                                                                                                                              | Beispiele                                              |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Klassen        | Der erste Buchstabe muss gross geschrieben werden. Vermeiden Sie Abkürzungen. Verwenden Sie einfache und aussagekräftige Namen.                                                                                     | <pre> class Image; class PrivilegedClient; </pre>      |
| Schnittstellen | Dieselben Regeln wie für die Klassen.                                                                                                                                                                               | <pre> interface SingleImage; interface Deposit; </pre> |
| Methoden       | Der Name der Methoden muss eine Aktion ausdrücken. Verwenden Sie wenn möglich Verben. Der erste Buchstabe wird klein geschrieben, der erste Buchstabe der darauffolgenden Wörter wird gross geschrieben.            | <pre> display(); getValue(); setValue(); </pre>        |
| Variablen      | Der erste Buchstabe wird klein geschrieben. Verwenden Sie bei Bedarf lange Namen (um der Verständlichkeit willen). Vermeiden Sie Namen mit nur einem Buchstaben, ausser für interne Zähler und temporäre Variablen. | <pre> int i; float width; String nameOfCaptain; </pre> |
| Konstanten     | Werden gross geschrieben. Das Trennzeichen wird unterstrichen.                                                                                                                                                      | <pre> final static int MAX_VALUE = 999; </pre>         |

## 7. Referenzen

### [1] Code-Konventionen von SUN

<http://java.sun.com/docs/codeconv/html/CodeConventionsTOC.doc.html>

### [2] Javadoc-Tool von SUN

<http://java.sun.com/j2se/javadoc/>